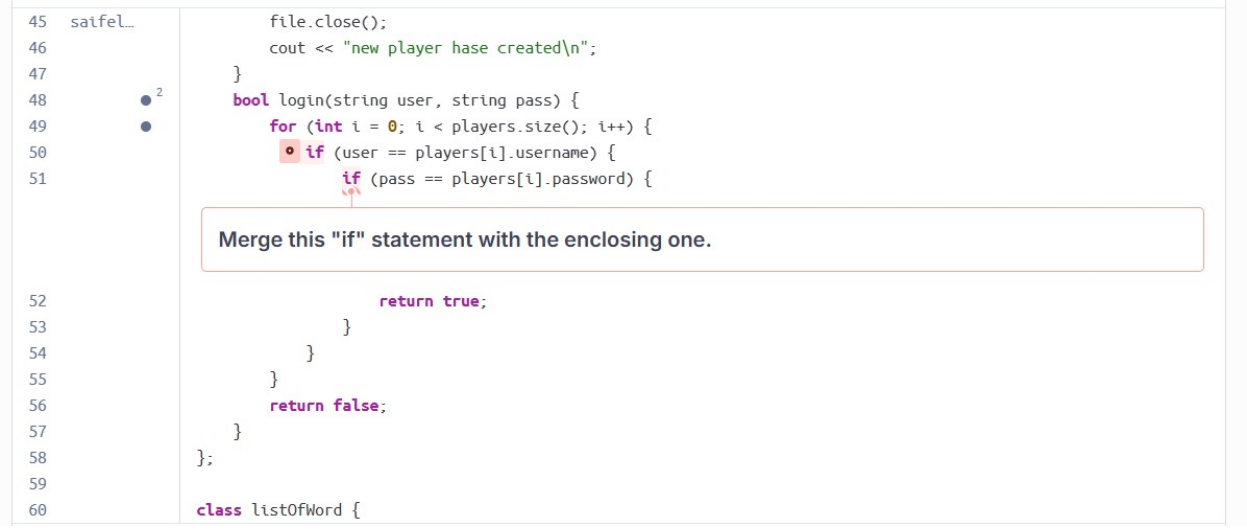
SonarQube Test




```
}
bool login(const string& user,const string& pass) {
    for (int i = 0; i < players.size(); i++) {
        if (user == players[i].username&& pass == players[i].password) {
                return true;
        }
    }
    return false;
}
```

1. What is the problem?

SonarQube detected nested if statements that could be combined into one condition.

2. How we solved it

We merged the two if statements into a single condition using the logical AND operator.

3. Why was it a problem?

Nested if statements made the code longer and less readable.

Combining them made the login check simpler and clearer.

```
class player {
public:
    string username, email, password;
    player() {};

    player(string name, string mail, string pass) {
```

Pass expensive to copy object "name" by reference to const.

```
        username = name;
        email = mail;
        password = pass;
    }
};
```

```
class player {
public:
    string username;
    string email;
    string password;
    player() = default ;

    player(const string& name,const string& mail,const string& pass) {
        username = name;
        email = mail;
        password = pass;
    }
};
```

1. What is the problem?

SonarQube detected that string parameters were passed by value, causing unnecessary copying of data.

2. How we solved it

We changed the constructor parameters to constant references so the strings are used directly without creating extra copies.

3. Why was it a problem?

Unnecessary copying increases memory usage and reduces performance, especially when many objects are created.

```
int main()
```

Refactor this function to reduce its Cognitive Complexity from 31 to the 25 allowed.

```
203             for (int i = 0; i < system.players.size(); i++) {
204                 if (name == system.players[i].username) {
205                     cout << "\nUsername already exists, chose another username: ";
206                     cin >> name;
207                     i = 0;
208                 }
209             }
210             system.signup(name, mail, pass);
211         }
212     }
213 }
```

```
void signup(string name , const string& mail ,const string& pass) {
    for (int i = 0; i < players.size(); i++) {
        if (name == players[i].username) {
            cout << "\nUsername already exists, chose another username: ";
            cin >> name;
            i = 0;
        }
    }
    player newplayer(name, mail, pass);
    players.push_back(newplayer);
    ofstream file("players.txt", ios::app);
    file <<endl << name << endl<< mail << endl<< pass;
    file.close();
    cout << "new player hase created\n";
}
```

1. What is the problem?

SonarQube detected high cognitive complexity in the main function due to too many nested conditions and operations.

2. How we solved it

We reduced the complexity by moving the signup logic into a separate signup function instead of keeping all the logic inside main.

3. Why was it a problem?

High cognitive complexity makes the code harder to read, understand, debug, and maintain.

Splitting the logic into smaller functions made the code cleaner and more organized.